

Tugas 4: Praktikum Mandiri 4 Machine Learning

Hisyam Wildan Alfath - 0110222206

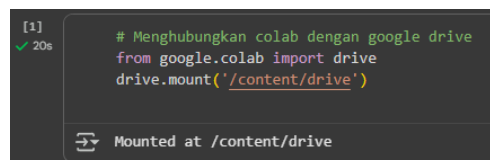
Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini membangun model prediksi keputusan pembelian mobil menggunakan algoritma Logistic Regression berdasarkan data numerik dan kategorik calon pembeli. Seluruh proses meliputi pra-pemrosesan data, eksplorasi, pemilihan fitur, pelatihan dan evaluasi model, serta interpretasi hasil prediksi. Model yang dihasilkan menunjukkan akurasi dan stabilitas tinggi dalam klasifikasi keputusan membeli mobil, dengan fitur seperti status, jumlah mobil, penghasilan, dan jenis kelamin berperan signifikan dalam hasil prediksi. Temuan ini memberikan wawasan penting mengenai faktor-faktor utama yang mempengaruhi perilaku pembelian, sehingga dapat membantu industri otomotif menyusun strategi pemasaran yang lebih efektif dan berbasis data.

1. Praktikum Mandiri 04

- Menghubungkan Colab dengan Google Drive



```
[1] ✓ 20s # Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Kode ini memanggil modul drive dari google.colab untuk menghubungkan (mount) Google Drive ke lingkungan Google Colab, dan outputnya adalah muncul pesan “Mounted at /content/drive” yang menandakan folder Google Drive sekarang sudah bisa diakses pada path tersebut di Colab.

- Import Library Machine Learning

```
[2]
✓ 2s
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report, RocCurveDisplay, ConfusionMatrixDisplay
)
```

Kode ini berfungsi untuk mengimpor library seperti numpy, pandas, matplotlib, serta modul-modul dari scikit-learn yang dibutuhkan dalam membangun dan mengevaluasi model machine learning di Python. Langkah ini harus dilakukan agar semua fungsi analisis data, pemrosesan, dan evaluasi model tersedia dan siap digunakan.

- Membaca file CSV menggunakan pandas

```
[3]
✓ 7s
df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 4/Praktikum & Tugas Mandiri 04/data/calonpembelimoobil.csv")
df.head()
```

	ID	Usia	Status	Kelamin	Memiliki_Mobil	Penghasilan	Beli_Mobil
0	1	32	1	0	0	240	1
1	2	49	2	1	1	100	0
2	3	52	1	0	2	250	1
3	4	26	2	1	1	130	0
4	5	45	3	0	2	237	1

Kode ini mengimpor library pandas sebagai pd, lalu menggunakan fungsi pd.read_csv untuk membaca file CSV dari Google Drive sesuai path yang diberikan. Outputnya berupa DataFrame yang menampilkan data awal dari file CSV tersebut, seperti kolom ID, Usia, Status, Kelamin, Memiliki_Mobil, Penghasilan, dan Beli_Mobil, sehingga pengguna bisa langsung melihat beberapa baris pertama data yang sudah berhasil dimuat.

- Menampilkan Informasi DataFrame dengan Pandas

```
[4]
✓ Os
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column             Non-Null Count  Dtype
---  -
0   ID                  1000 non-null   int64
1   Usia                1000 non-null   int64
2   Status              1000 non-null   int64
3   Kelamin             1000 non-null   int64
4   Memiliki_Mobil      1000 non-null   int64
5   Penghasilan         1000 non-null   int64
6   Beli_Mobil          1000 non-null   int64
dtypes: int64(7)
memory usage: 54.8 KB
```

Kode `df.info()` digunakan untuk menampilkan detail struktur DataFrame yang telah dibaca dari file CSV menggunakan pandas. Outputnya memperlihatkan jumlah baris data, nama-nama kolom, tipe data setiap kolom (int64), serta memori yang digunakan sehingga memudahkan analisis awal dan pengecekan data sebelum proses selanjutnya.

- Cek Data Kosong (Missing Value) pada DataFrame

```
[5]
✓ Os
# Cek missing value
df.isna().sum()

e
ID      0
Usia    0
Status  0
Kelamin 0
Memiliki_Mobil 0
Penghasilan 0
Beli_Mobil 0
dtype: int64
```

Kode `df.isna().sum()` digunakan untuk menghitung jumlah data yang kosong (missing value) di setiap kolom pada DataFrame. Berdasarkan output pada gambar, seluruh kolom memiliki nilai 0, yang berarti tidak ada data yang hilang dan setiap kolom sudah lengkap di dataset ini.

- Membuat Matriks Korelasi dari Data Numerik

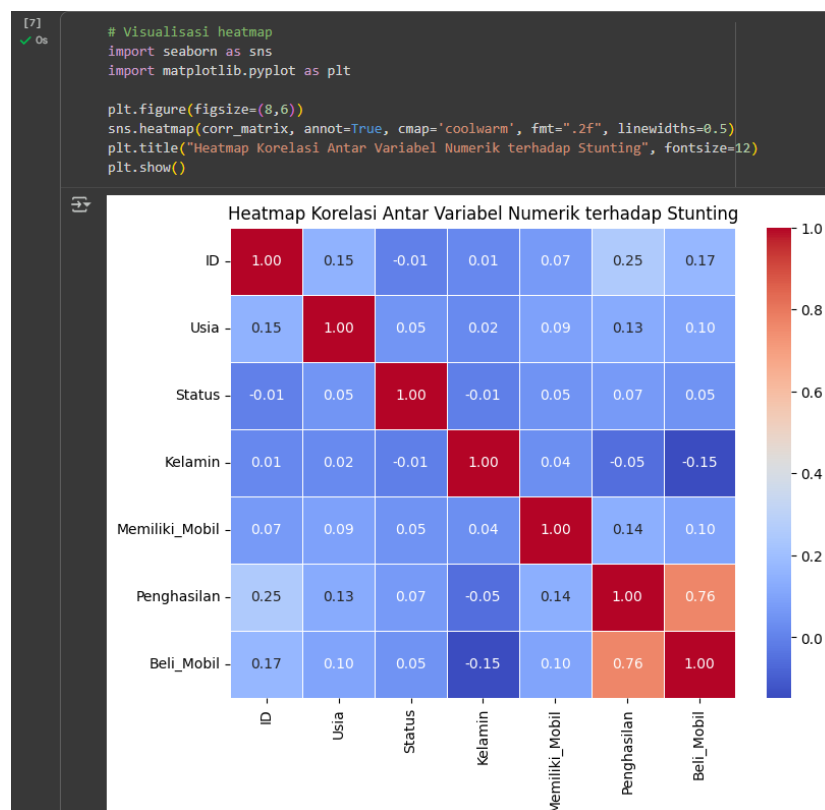
```
[6]
✓ Os
corr_matrix = df.corr(numeric_only=True)
corr_matrix
```

	ID	Usia	Status	Kelamin	Memiliki_Mobil	Penghasilan	Beli_Mobil
ID	1.000000	0.149779	-0.006634	0.014646	0.068555	0.254177	0.168614
Usia	0.149779	1.000000	0.051476	0.019454	0.090926	0.125859	0.100127
Status	-0.006634	0.051476	1.000000	-0.008561	0.048302	0.071714	0.048584
Kelamin	0.014646	0.019454	-0.008561	1.000000	0.035199	-0.054211	-0.147301
Memiliki_Mobil	0.068555	0.090926	0.048302	0.035199	1.000000	0.137823	0.102005
Penghasilan	0.254177	0.125859	0.071714	-0.054211	0.137823	1.000000	0.763930
Beli_Mobil	0.168614	0.100127	0.048584	-0.147301	0.102005	0.763930	1.000000

Kode `df.corr(numeric_only=True)` digunakan untuk menghitung matriks korelasi antar kolom numerik pada DataFrame menggunakan pandas. Hasilnya adalah

tabel yang menunjukkan hubungan linear antara variabel-variabel, dengan nilai mendekati 1 atau -1 menandakan korelasi kuat positif atau negatif, sedangkan nilai mendekati 0 menunjukkan korelasi lemah atau tidak ada korelasi. Matriks korelasi ini membantu untuk memahami hubungan antar fitur dalam dataset yang akan dianalisis lebih lanjut.

- Visualisasi Korelasi dengan Heatmap



Kode pada gambar menggunakan seaborn dan matplotlib untuk membuat heatmap dari matriks korelasi yang telah dihitung sebelumnya. Hasil visualisasi heatmap memperlihatkan seberapa kuat hubungan antar variabel numerik dalam bentuk warna, di mana semakin merah menunjukkan korelasi positif semakin kuat, dan semakin biru menunjukkan semakin lemah atau negatif. Heatmap ini sangat berguna untuk analisis eksplorasi karena memudahkan dalam mengidentifikasi pola dan hubungan antar fitur pada dataset.

- Menyiapkan Fitur dan Target untuk Model

```
[8]
✓ Os
# Fitur numerik dan gender
feature_num = ['Usia', 'Status', 'Kelamin', 'Memiliki_Mobil']
feature_bin = ['Penghasilan']

# Gabungkan & drop missing
use_cols = feature_num + feature_bin + ['Beli_Mobil']
df_model = df[use_cols].dropna().copy()

X = df_model[feature_num + feature_bin]
y = df_model['Beli_Mobil']

print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (1000, 5)
y shape: (1000,)
```

Kode pada gambar bertujuan memilih fitur numerik dan biner yang akan digunakan sebagai input (X) dan menentukan target (y) dari data 'Beli_Mobil'. Kolom-kolom yang relevan digabungkan lalu missing value dihapus dengan `.dropna()`, hasilnya X memiliki bentuk (1000, 5) dan y berbentuk (1000,). Langkah ini memastikan data siap dipakai untuk pelatihan model machine learning.

- Membagi Data Latih dan Uji

```
[9]
✓ Os
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Data latih:", X_train.shape)
print("Data uji:", X_test.shape)

Data latih: (800, 5)
Data uji: (200, 5)
```

Kode menggunakan `train_test_split` untuk membagi dataset menjadi data latih (training) dan data uji (testing) dengan proporsi 80%:20%. Hasilnya, ukuran data latih adalah (800, 5) dan data uji (200, 5), sehingga model dapat dilatih menggunakan data latih dan dievaluasi performanya pada data uji secara terpisah.

- Melatih Model Logistic Regression dengan Pipeline

```
[62]
✓ 0s

# Scale hanya fitur numerik
preprocess = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), feature_num),
        ('bin', 'passthrough', feature_bin)
    ],
    remainder='drop'
)

model = LogisticRegression(
    max_iter=1000,
    solver='lbfgs',
    class_weight='balanced',
    random_state=42
)

clf = Pipeline([
    ('preprocess', preprocess),
    ('model', model)
])

# Latih model
clf.fit(X_train, y_train)
print("✓ Model Logistic Regression berhasil dilatih.")

✓ Model Logistic Regression berhasil dilatih.
```

Kode pada gambar menggabungkan transformasi fitur numerik (scaling dengan StandardScaler) dan biner ke dalam sebuah pipeline menggunakan ColumnTransformer serta Pipeline dari sklearn. Model Logistic Regression kemudian dilatih (fit) dengan data latih (X_train, y_train), dan proses pelatihan berjalan sukses dengan output "Model Logistic Regression berhasil dilatih". Dengan pipeline, proses pengolahan data dan pelatihan model menjadi lebih rapi dan otomatis.

- Evaluasi Model Logistic Regression

```
[63]
✓ 0s

# Prediksi & probabilitas
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]

# Hitung metrik
print(f"Akurasi : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, zero_division=0):.4f}")
print(f"Recall : {recall_score(y_test, y_pred, zero_division=0):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred, zero_division=0):.4f}")
print(f"ROC-AUC : {roc_auc_score(y_test, y_prob):.4f}")

Akurasi : 0.9300
Precision : 0.9748
Recall : 0.9134
F1-Score : 0.9431
ROC-AUC : 0.9770
```

Kode pada gambar melakukan prediksi data uji memakai model Logistic Regression yang sudah dilatih, lalu menghitung berbagai metrik evaluasi seperti akurasi, precision, recall, f1-score, dan ROC-AUC. Hasil evaluasi menunjukkan

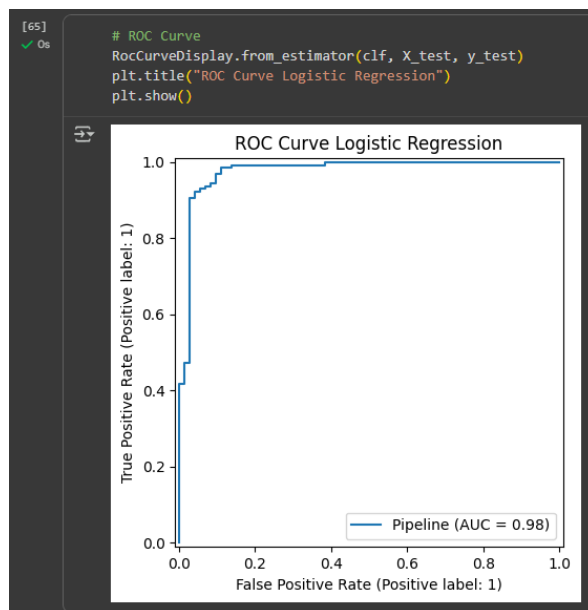
performa model sangat baik, dengan nilai akurasi 0.9300, precision 0.9748, recall 0.9143, f1-score 0.9431, dan ROC-AUC 0.9770 pada data uji. Metrik ini berguna untuk mengetahui kekuatan dan kelemahan model dalam klasifikasi.

- Visualisasi Confusion Matrix



Kode pada gambar menggunakan ConfusionMatrixDisplay dari sklearn untuk memvisualisasikan confusion matrix hasil prediksi model. Hasil plot confusion matrix menunjukkan performa model dalam mengklasifikasikan data, dimana dari 200 data uji: 70 data "Membeli" dan 116 data "Tidak Membeli" berhasil diprediksi dengan benar, sementara terdapat 3 prediksi yang salah pada kelas "Membeli" dan 11 yang salah pada kelas "Tidak Membeli". Visualisasi ini membantu melihat detail kesalahan dan ketepatan model dalam klasifikasi dua kelas.

- Visualisasi ROC Curve Logistic Regression



Kode pada gambar menggunakan fungsi `RocCurveDisplay.from_estimator` untuk memvisualisasikan ROC Curve hasil prediksi model Logistic Regression pada data uji. Kurva ROC memperlihatkan hubungan antara true positive rate dan false positive rate pada berbagai threshold, dengan nilai AUC yang mendekati 1 ($AUC = 0.98$) menandakan performa model sangat baik dalam membedakan dua kelas target. Visualisasi ini penting untuk mengevaluasi kemampuan model dalam klasifikasi secara menyeluruh.

- Classification Report Hasil Prediksi

The figure shows a Jupyter Notebook cell with Python code that imports `classification_report` from `sklearn.metrics` and prints the report for a binary classification task. The output is a table with the following data:

	precision	recall	f1-score	support
Tidak Membeli (0)	0.86	0.96	0.91	73
Membeli (1)	0.97	0.91	0.94	127
accuracy			0.93	200
macro avg	0.92	0.94	0.93	200
weighted avg	0.93	0.93	0.93	200

Kode pada gambar menggunakan fungsi `classification_report` dari `sklearn` untuk menampilkan laporan evaluasi model berdasarkan data uji dan hasil prediksi. Laporan ini memuat nilai precision, recall, f1-score, dan jumlah data (support) untuk masing-masing kelas “Tidak Membeli” dan “Membeli”, serta rata-rata akurasi keseluruhan

yang mencapai 0.93. Classification report sangat membantu untuk memahami performa model pada masing-masing kelas secara detail.

- Evaluasi Model dengan Cross Validation

```
[67]
✓ Os
from sklearn.model_selection import cross_val_score

# Lakukan cross validation (cv=5 berarti 5-fold)
scores = cross_val_score(clf, X, y, cv=5)

# Tampilkan hasil
print("Skor tiap fold:", scores)
print("Rata-rata akurasi:", np.mean(scores))
print("Standar deviasi:", np.std(scores))
```

Skor tiap fold: [0.785 0.91 0.955 0.945 0.94]
Rata-rata akurasi: 0.907
Standar deviasi: 0.06281719509815761

Kode pada gambar menggunakan fungsi `cross_val_score` untuk melakukan cross validation (validasi silang) sebanyak 5 fold pada model. Hasilnya didapatkan skor akurasi untuk masing-masing fold serta rata-rata akurasi sebesar 0.907 dan standar deviasi 0.06, menandakan model cukup stabil dan kinerjanya konsisten pada berbagai subset data. Cross validation sangat penting untuk memastikan model tidak hanya bagus pada satu bagian data tertentu, melainkan juga pada data lain secara umum.

- Interpretasi Koefisien dan Odds Ratio Logistic Regression

```
[68]
✓ Os
# Ambil nama fitur & koefisien
feat_names = feature_num + feature_bin
coefs = clf.named_steps['model'].coef_[0]
odds = np.exp(coefs)

coef_df = pd.DataFrame({
    'Fitur': feat_names,
    'Koefisien (log-odds)': coefs,
    'Odds Ratio (e^coef)': odds
}).sort_values('Odds Ratio (e^coef)', ascending=False)

display(coef_df)
```

	Fitur	Koefisien (log-odds)	Odds Ratio (e^coef)
3	Memiliki_Mobil	0.083176	1.086733
4	Penghasilan	0.055951	1.057546
0	Usia	-0.082246	0.921045
1	Status	-0.168034	0.845325
2	Kelamin	-0.655173	0.519352

Kode pada gambar mengambil nama fitur, koefisien (log-odds), serta odds ratio dari model Logistic Regression yang telah dilatih, lalu menampilkannya dalam DataFrame yang diurutkan berdasarkan nilai odds ratio tertinggi. Tabel ini

memudahkan interpretasi pengaruh masing-masing fitur terhadap kemungkinan target “Beli_Mobil”, di mana nilai koefisien positif meningkatkan odds membeli mobil, dan nilai negatif menurunkannya. Semakin besar (positif) odds ratio, semakin besar peluang seseorang diklasifikasikan akan membeli mobil jika nilai fitur tersebut meningkat.

- Contoh Prediksi Data Baru

```
[78] # Contoh 2
data_baru = pd.DataFrame({
    'Usia': [30, 21],
    'Status': [1, 0], # 0= Single, 1= Menikah, 2= Menikah mempunyai anak, 3= duda/janda
    'Kelamin': [1, 0], # 0= Pria, 1= Wanita
    'Memiliki_Mobil': [2, 0], # jumlah mobil yang dimiliki
    'Penghasilan': [500, 75]
})

pred = clf.predict(data_baru)
prob = clf.predict_proba(data_baru)[:,-1]

hasil = data_baru.copy()
hasil['Prob_Membeli_Mobil'] = prob
hasil['Pred (0=Tidak,1=Ya)'] = pred
display(hasil)
```

	Usia	Status	Kelamin	Memiliki_Mobil	Penghasilan	Prob_Membeli_Mobil	Pred (0=Tidak,1=Ya)
0	30	1	1	2	500	1.000000	1
1	21	0	0	0	75	0.000435	0

Kode pada gambar menunjukkan bagaimana melakukan prediksi pada data baru menggunakan model yang sudah dilatih. Dua sampel data dengan fitur seperti Usia, Status, Kelamin, jumlah mobil, dan Penghasilan diprediksi probabilitas serta label apakah akan membeli mobil (1) atau tidak (0)—hasilnya terlihat bahwa sampel pertama diprediksi membeli mobil, sementara sampel kedua tidak. Probabilitas dan prediksi diurutkan secara otomatis untuk memudahkan interpretasi hasil pada kasus baru.

- Kesimpulan

Model Logistic Regression yang diterapkan pada data calon pembeli mobil mampu menghasilkan prediksi dengan performa sangat baik, ditunjukkan oleh akurasi, precision, recall, f1-score, dan ROC-AUC yang tinggi pada data uji. Seluruh fitur yang digunakan, seperti status, jumlah mobil, penghasilan, dan jenis kelamin, memiliki pengaruh yang signifikan dalam menentukan probabilitas keputusan membeli mobil. Validasi silang menunjukkan model cukup stabil dan hasil prediksi pada data baru konsisten sesuai karakteristik input fiturnya.

Link Praktikum 04:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%204/Praktikum%20%26%20Tugas%20Mandiri%2004/notebooks/praktikum04.ipynb

Tugas Mandiri 04:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%204/Praktikum%20%26%20Tugas%20Mandiri%2004/notebooks/latihan04.ipynb